МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
<БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ>

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №4

Специальность АС-67

Выполнил
Масюк А. М.,
студент группы АС-67

Проверил
А. А. Крощенко,
ст. преп. кафедры ИИТ,
«___> _____2026 г.

Брест 2026

Цель: построить, обучить и оценить многослойный перцептрон (MLP) для решения задачи классификации

Общий план для всех вариантов:

1. Импорт библиотек и подготовка данных

• импортируйте torch, torch.nn, torch.optim, а также sklearn для загрузки данных и их предобработки;

• загрузите датасет, выполните стандартизацию (StandardScaler) и кодирование признаков;

• разделите данные на обучающую и тестовую выборки;

• преобразуйте данные (признаки и метки) в тензоры PyTorch: torch.tensor(X_train, dtype=torch.float32).

2. Определение архитектуры нейронной сети

• создайте класс, наследуемый от torch.nn.Module;

• в методе __init__ определите все слои, которые будете использовать (например, nn.Linear, nn.ReLU, nn.Dropout);

• в методе forward опишите последовательность применения слоев к входным данным.

3. Инициализация модели, функции потерь и оптимизатора

• создайте экземпляр вашей модели: model = MLP();

• определите функцию потерь. Для бинарной классификации используйте nn.BCEWithLogitsLoss, для многоклассовой – nn.CrossEntropyLoss;

• определите оптимизатор: optimizer = torch.optim.Adam(model.parameters(), lr=0.001).

4. Написание цикла обучения (Training Loop)

• запустите цикл на определенное количество эпох;

• внутри цикла:

1. переведите модель в режим обучения: model.train();
2. сделайте предсказание (forward pass):
y_pred = model(X_train);
3. рассчитайте потери (loss): loss = criterion(y_pred, y_train);
4. обнулите градиенты: optimizer.zero_grad();
5. выполните обратное распространение ошибки: loss.backward();
6. сделайте шаг оптимизации: optimizer.step().

5. Оценка модели (Evaluation)

• переведите модель в режим оценки: model.eval();.

• используйте with torch.no_grad():, чтобы отключить расчетградиентов;

• сделайте предсказания на тестовых данных;

• преобразуйте выходные данные (логиты) в предсказанные классы (например, с помощью torch.argmax или проверки порога > 0);

• рассчитайте метрики (accuracy, f1-score и т.д.), используя sklearn.metrics**.**

Вариант 6 Диагностика диабета

• Pima Indians Diabetes

• Задача: предсказать наличие диабета (бинарная классификация).

• Архитектура:

o входной слой;

o два скрытых слоя: первый с 12 нейронами, второй с 8 (ReLU);

o выходной слой с 1 нейроном (Sigmoid).

• Эксперимент: удалите второй скрытый слой (оставив один с 12 нейронами). Сравните recall для класса "диабет" в обеих моделях.

Код программы:

```
import os
import torch
from torch import nn
from torch import optim
```

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, recall_score
# Выбор устройства
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Платформа обучения:", device)
TARGET = "Outcome"
COLUMNS = [
    "Pregnancies",
    "Glucose",
    "BloodPressure",
    "SkinThickness",
    "Insulin",
    "BMI",
    "DiabetesPedigreeFunction",
    "Age",
    TARGET
]
EPOCHS = 500
# Класс многослойного перцептрона
class MLP(nn.Module):
    def __init__(self, in_features, two_hidden=True):
        super().__init__()

        if two_hidden:
            self.layers = nn.Sequential(
                nn.Linear(in_features, 12),
                nn.ReLU(),

                nn.Linear(12, 8),
                nn.ReLU(),

                nn.Linear(8, 1)
            )
        else:
            self.layers = nn.Sequential(
                nn.Linear(in_features, 12),
                nn.ReLU(),

                nn.Linear(12, 1)
            )
    def forward(self, x):
        return self.layers(x)
# Загрузка и подготовка данных
def load_data():
    # Путь к файлу
    base_dir = os.path.dirname(os.path.abspath(__file__))
    file_path = os.path.join(
        base_dir,
        "pima-indians-diabetes.csv"
    )
    if not os.path.exists(file_path):
        raise FileNotFoundError(
            f"Файл не найден: {file_path}"
        )
    # Загрузка датасета
    df = pd.read_csv(
        file_path,
        header=None,
        names=COLUMNS,
        comment="#",
        sep=","
```

```
    )
    # Преобразование всех столбцов в числа
    for column in COLUMNS:
        df[column] = pd.to_numeric(
            df[column],
            errors="coerce"
        )
    # Удаление строк без целевого значения
    df = df.dropna(subset=[TARGET])
    # Оставляем только классы 0 и 1
    df = df[df[TARGET].isin([0, 1])]
    # Удаление полностью пустых строк
    df = df.dropna(how="all")
    print("\nРазмер датасета:", df.shape)
    # Разделение признаков и целевой переменной
    X = df.drop(columns=[TARGET])
    y = df[TARGET].astype(np.float32)
    # Разделение на обучающую и тестовую выборки
    X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.25,
        random_state=42,
        stratify=y
    )
    # Замена нулевых значений на пропуски
    zero_columns = [
        "Glucose",
        "BloodPressure",
        "SkinThickness",
        "Insulin",
        "BMI"
    ]
    X_train[zero_columns] = X_train[zero_columns].replace(
        0, np.nan
    )
    X_test[zero_columns] = X_test[zero_columns].replace(
        0, np.nan
    )
    # Заполнение пропусков медианами
    medians = X_train.median()
    X_train = X_train.fillna(medians)
    X_test = X_test.fillna(medians)
    # Стндартизация признаков
    scaler = StandardScaler()
    X_train = scaler.fit_transform(X_train)
    X_test = scaler.transform(X_test)
    # Преобразование в тензоры PyTorch
    X_train = torch.tensor(
        X_train,
        dtype=torch.float32
    ).to(device)
    X_test = torch.tensor(
        X_test,
        dtype=torch.float32
    ).to(device)
    y_train = torch.tensor(
        y_train.to_numpy(),
        dtype=torch.float32
    ).unsqueeze(1).to(device)
    y_test = y_test.to_numpy().astype(int)
    return X_train, X_test, y_train, y_test
# Обучение и оценка модели
def train_and_evaluate(two_hidden, data):
```

```
    X_train, X_test, y_train, y_test = data
    torch.manual_seed(42)
    model = MLP(
        in_features=X_train.shape[1],
        two_hidden=two_hidden
    ).to(device)
    # Функция потерь
    criterion = nn.BCEWithLogitsLoss()
    # Оптимизатор
    optimizer = optim.Adam(
        model.parameters(),
        lr=0.001
    )
    print("\nАрхитектура модели:")
    print(model)
    print("\nКоличество эпох:", EPOCHS)
    print("Оптимизатор: Adam")
    print("Функция потерь: BCEWithLogitsLoss")
    # Цикл обучения
    for epoch in range(EPOCHS):
        model.train()
        y_pred = model(X_train)
        loss = criterion(y_pred, y_train)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        if (epoch + 1) % 100 == 0:
            print(
                f"Эпоха {epoch + 1}/{EPOCHS}, "
                f"ошибка: {loss.item():.4f}"
            )
    # Оценка модели
    model.eval()
    with torch.no_grad():
        logits = model(X_test)
        probabilities = torch.sigmoid(logits)
        predictions = (
            probabilities >= 0.5
        ).int().squeeze(1).cpu().numpy()
    print("\nОтчет классификации:")
    print(
        classification_report(
            y_test,
            predictions,
            labels=[0, 1],
            target_names=[
                "Нет диабета (0)",
                "Диабет (1)"
            ],
            zero_division=0
        )
    )
    recall = recall_score(
        y_test,
        predictions,
        pos_label=1,
        zero_division=0
    )
    print(
        f"Recall для класса диабет: {recall:.4f}"
    )
    return recall
# Основная функция
def solution():
```

```
    data = load_data()
    # Первая модель
    print("\n" + "=" * 50)
    print("МОДЕЛЬ 1: ДВА СКРЫТЫХ СЛОЯ (12 И 8)")
    print("=" * 50)
    recall_two = train_and_evaluate(
        two_hidden=True,
        data=data
    )
    # Вторая модель
    print("\n" + "=" * 50)
    print("МОДЕЛЬ 2: ОДИН СКРЫТЫЙ СЛОЙ (12)")
    print("=" * 50)
    recall_one = train_and_evaluate(
        two_hidden=False,
        data=data
    )
    # Сравнение моделей
    print("\n" + "=" * 50)
    print("СРАВНЕНИЕ РЕЗУЛЬТАТОВ")
    print("=" * 50)
    print(
        f"Два скрытых слоя (12, 8): {recall_two:.4f}"
    )
    print(
        f"Один скрытый слой (12): {recall_one:.4f}"
    )
    if recall_two > recall_one:
        print(
            "\nПредпочтительнее модель с двумя скрытыми слоями."
        )
    elif recall_one > recall_two:
        print(
            "\nПредпочтительнее модель с одним скрытым слоем."
        )
    else:
        print("\nОбе модели показали одинаковый Recall.")
    print("=" * 50)
if __name__ == "__main__":
    solution()
```

```
Платформа обучения: cpu

Размер датасета: (768, 9)

================================================
МОДЕЛЬ 1: ДВА СКРЫТЫХ СЛОЯ (12 И 8)
================================================

Архитектура модели:
MLP(
  (layers): Sequential(
    (0): Linear(in_features=8, out_features=12, bias=True)
    (1): ReLU()
    (2): Linear(in_features=12, out_features=8, bias=True)
    (3): ReLU()
    (4): Linear(in_features=8, out_features=1, bias=True)
  )
)

Количество эпох: 500
Оптимизатор: Adam
Функция потерь: BCEWithLogitsLoss
Эпоха 100/500, ошибка: 0.5635
Эпоха 200/500, ошибка: 0.4513
Эпоха 300/500, ошибка: 0.4109
Эпоха 400/500, ошибка: 0.3866
Эпоха 500/500, ошибка: 0.3679

Отчет классификации:
                precision    recall  f1-score   support

Нет диабета (0)       0.79      0.82      0.80       125
     Диабет (1)       0.63      0.60      0.62        67

       accuracy                           0.74       192
      macro avg       0.71      0.71      0.71       192
   weighted avg       0.74      0.74      0.74       192

Recall для класса диабет: 0.5970
```

```
================================================
МОДЕЛЬ 2: ОДИН СКРЫТЫЙ СЛОЙ (12)
================================================

Архитектура модели:
MLP(
  (layers): Sequential(
    (0): Linear(in_features=8, out_features=12, bias=True
    (1): ReLU()
    (2): Linear(in_features=12, out_features=1, bias=True
  )
)

Количество эпох: 500
Оптимизатор: Adam
Функция потерь: BCEWithLogitsLoss
Эпоха 100/500, ошибка: 0.5572
Эпоха 200/500, ошибка: 0.4752
Эпоха 300/500, ошибка: 0.4434
Эпоха 400/500, ошибка: 0.4277
Эпоха 500/500, ошибка: 0.4199

Отчет классификации:
                precision    recall  f1-score   support

Нет диабета (0)       0.76      0.82      0.79       125
     Диабет (1)       0.61      0.51      0.55        67

       accuracy                           0.71       192
      macro avg       0.68      0.67      0.67       192
   weighted avg       0.70      0.71      0.71       192

Recall для класса диабет: 0.5075

================================================
СРАВНЕНИЕ РЕЗУЛЬТАТОВ
================================================
Два скрытых слоя (12, 8): 0.5970
Один скрытый слой (12): 0.5075

Предпочтительнее модель с двумя скрытыми слоями.
================================================
```